CELERY CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & BROKER

Installation

```
pip install celery redis
# celery.py
from celery import Celery
app = Celery('myapp',
broker='redis://localhost:6379/0',
backend='redis://localhost:6379/1')
celery -A myapp worker -l info
```

02. DEFINING TASKS

Architecture

```
@app.task(bind=True, max_retries=3)
def send_email(self, to_addr, subject):
try:
mailer.send(to_addr, subject)
except Exception as exc:
raise self.retry(exc=exc, countdown=60)
# Simple task:
@app.task
def add(x, y): return x + y
```

03. CALLING TASKS

YAML / Config

```
add.delay(4, 4)          # fire and forget
add.apply_async((4,4), countdown=30) # delayed
add.apply_async((4,4), eta=datetime.utcnow()+...
result = add.delay(4,4)
result.get(timeout=10)   # blocks until done
result.status  # PENDING / STARTED / SUCCESS ...
```

04. RESULTS & STATE

Workflow

```
res = add.apply_async((2,3))
res.id             # task UUID
res.ready()        # True if done
res.successful()   # True if SUCCESS
res.result         # return value
res.traceback      # if FAILURE
AsyncResult('uuid').get()  # by task ID
```

05. PERIODIC TASKS (BEAT)

CRUD API

```
from celery.schedules import crontab
app.conf.beat_schedule = {
'daily-report': {
'task': 'tasks.send_report',
'schedule': crontab(hour=8, minute=0),
},
'every-30s': {
'task': 'tasks.heartbeat',
'schedule': 30.0,
},
}
celery -A myapp beat -l info
```

06. RETRIES

Integration

```
@app.task(bind=True, autoretry_for=(Exception...
retry_kwargs={'max_retries': 5},
retry_backoff=True,
retry_backoff_max=700,
retry_jitter=True)
def fetch_data(self, url): ...
self.retry(exc=exc, countdown=2**self.request...
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. CANVAS: CHAIN & GROUP

Hardening

```
from celery import chain, group, chord
# Chain (sequential):
result = chain(task1.s(arg), task2.s(), task3...
# Group (parallel):
job = group(add.s(i, i) for i in range(10))
result = job.apply_async()
# Chord (parallel then callback):
chord(group(tasks))(callback.s())
```

08. WORKER CLI

Observability

```
celery -A proj worker --concurrency=4 -l info
celery -A proj worker -P gevent -c 1000
celery -A proj worker --queues=email,high
celery -A proj inspect active
celery -A proj purge  # clear all queued tasks
celery -A proj events  # real-time monitoring
celery -A proj flower  # web monitoring UI
```

09. ROUTING & QUEUES

Optimization

```
@app.task(queue='high_priority')
def urgent_task(): ...
app.conf.task_routes = {
'tasks.urgent_*': {'queue': 'high_priority'},
'tasks.email_*': {'queue': 'email'},
}
celery worker -Q high_priority,email
```

10. CONFIGURATION

Unit Tests

```
app.conf.update(
task_serializer='json',
accept_content=['json'],
result_expires=3600,
task_time_limit=300,      # hard time limit
task_soft_time_limit=240, # soft (raises Soft...
worker_max_tasks_per_child=1000,
)
```

11. SIGNALS

Commands

```
from celery.signals import task_success, task...
@task_success.connect
def on_success(result, **kwargs):
metrics.incr('task.success')
@task_failure.connect
def on_failure(exception, **kwargs):
alert.send(str(exception))
```

Signals: prerun, postrun, retry, revoke

12. COMMON GOTCHAS

Warning

Don't pass ORM objects to tasks pass IDs instead
result.get() inside another task causes deadlock
Pickle serializer security risk use JSON
Task imports must be accessible in worker environment
Beat scheduler needs single instance (use RedBeat for HA)